EXP:2 - LINEAR PROGRAMMING PROBLEM- CONSTRAINT OPTIMIZATION

AIM:

To solve a Linear Programming Problem (LPP) for constraint optimization using Python.

PROCEDURE:

1. Install Required Libraries:

- Make sure you have numpy and scipy installed. You can install them using pip if not already installed.

  bash

  Copy code

  pip install numpy scipy

2. Define the Problem:

- Identify the objective function to maximize or minimize.
- Determine the constraints (equality and inequality).

3. Set Up the Problem in Python:

- Use the scipy.optimize.linprog function to set up and solve the LPP.

4. Run the Code:

- Execute the Python code to get the optimized result.

5. Analyze the Output:

- Interpret the results and validate the solution against the problem constraints.

CODE:

1.a) Code to solve a Linear Programming Problem using Python.

```
import numpy as np

from scipy.optimize import linprog


# Cost matrix (3 warehouses × 4 stores)

cost = np.array([

    [8, 6, 10, 9],

    [9, 7, 4, 2],

    [3, 4, 2, 5]

])
```

```
# Flatten cost matrix
c = cost.flatten()

# Supply and demand
supply = np.array([20, 30, 25])
demand = np.array([10, 25, 20, 20])

# Create equality constraints
A_eq = []
b_eq = []

# Supply constraints
for i in range(len(supply)):
    constraint = [0] * len(c)
    for j in range(len(demand)):
        constraint[i * len(demand) + j] = 1
    A_eq.append(constraint)
    b_eq.append(supply[i])

# Demand constraints
for j in range(len(demand)):
    constraint = [0] * len(c)
    for i in range(len(supply)):
        constraint[i * len(demand) + j] = 1
    A_eq.append(constraint)
    b_eq.append(demand[j])

# Solve using linprog
result = linprog(c, A_eq=A_eq, b_eq=b_eq, bounds=(0, None), method='highs')
```

```
# Reshape result
result_matrix = result.x.reshape(cost.shape)

# Output
print("Optimal cost:", result.fun)
print("Optimal transportation plan:")
print(result_matrix)
```

1.a) Output:

```
Optimal cost: 270.0
Optimal transportation plan:
[[ 0. 20.  0.  0.]
 [ 0.  0. 10. 20.]
 [10.  5. 10.  0.]]
```

1.b) Code to solve a Linear Programming Problem using Python in Graphical Method.

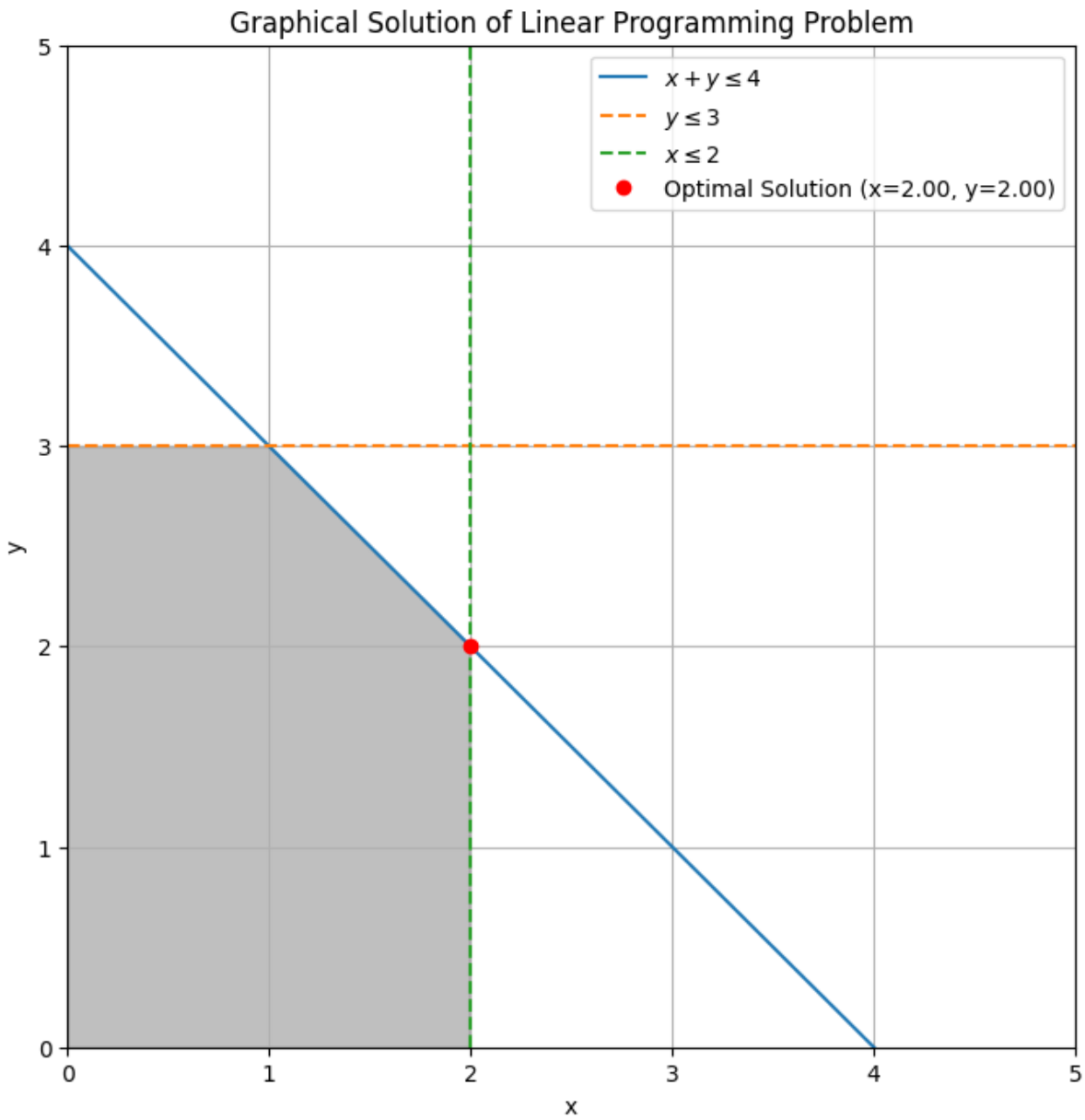
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import linprog

# Objective function coefficients (maximize Z = 3x + 2y)
c = [-3, -2]    # Coefficients are negated for linprog as it does minimization

# Coefficients for the inequality constraints
A = [[1,1],
     [1,0],
     [0,1]]

# Right-hand side values for the inequality constraints
b = [4, 2, 3]

# Bounds for the variables x and y
```

```
x_bounds = (0, None)
y_bounds = (0, None)

# Solve the linear programming problem
res = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds], method = 'highs')

# Extract the optimal solution
x_opt, y_opt = res.x
z_opt = -res.fun  # Negate the result to get the maximum value

print(f"Optimal solution: x = {x_opt}, y = {y_opt}, z = {z_opt}")

# Plotting the constraints
x = np.linspace(0, 5, 400)
y1 = 4 - x
y2 = np.ones_like(x) * 3
y3 = 2 * np.ones_like(x)

plt.figure(figsize=(8,8))

# Plot each constraint
plt.plot(x, y1, label=r'$x + y \leq 4$')
plt.plot(x, y2, label=r'$y \leq 3$', linestyle='--')
plt.plot(y3, x, label=r'$x \leq 2$', linestyle='--')

# Fill each constraint
plt.fill_between(x, np.minimum(y1, y2), where=(x <= 2), color='grey', alpha=0.5)

# Highlight the optimal solution point
plt.plot(x_opt, y_opt, 'ro', label=f'Optimal Solution (x={x_opt:.2f}, y={y_opt:.2f})')
```

```
plt.xlim(0, 5)
plt.ylim(0, 5)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Graphical Solution of Linear Programming Problem')
plt.legend()
plt.grid(True)
plt.show()
```

1.b) Output:

Optimal solution: x = 2.0, y = 2.0, z = 10.0



Result:
The given transportation problem was solved successfully using Python, and the optimal transportation cost and shipment plan were obtained.